

StrategAI: Integrating Large Language Models and Diffusion Transformers for AI-Driven Strategy Game Development

Sivert Asakskogen

*Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
sas054@uit.no*

Max Hovding

*Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
mho251@uit.no*

Abstract—Strategy game development faces two persistent bottlenecks: AI behavior authoring, where hand-crafted decision systems become brittle and labor-intensive as complexity grows, and asset creation, where manual pixel art production scales poorly with game scope. This paper presents StrategAI, a full-stack strategy game that addresses both challenges by integrating three generative AI modalities. Three AI civilizations—Mongolia under Genghis Khan, Egypt under Cleopatra, and India under Gandhi—are controlled by GPT-5.4-mini via OpenAI’s tool-use API. Each AI emits high-level strategic intents through a novel abstraction layer that decouples LLM reasoning from deterministic game mechanics, preventing direct game-state manipulation while preserving strategic agency. Concurrently, a self-hosted asset server generates top-down medieval pixel-art assets on demand using FLUX.2 Klein 4B Distilled, a 4-billion-parameter Diffusion Transformer, orchestrated through ComfyUI with a four-layer prompt architecture separating workflow configuration, style constraints, semantic descriptions, and assembly logic. A custom-trained LoRA adapter, produced through a systematic 3×2 experiment matrix evaluating caption detail against rank configuration on a curated 100-image dataset, enforces consistent top-down perspective and medieval architectural styling via a trigger token. The system degrades gracefully through four fallback tiers when AI services are unavailable. Informally observed generation times of 2.5–6 seconds on a Blackwell RTX 6000 and 3.5–7 seconds on an RTX 3090 suggest workstation GPUs can support near-interactive generative asset pipelines, and an OpenAI TTS pipeline that produces an epic voiced narration at game start. We detail the architectural decisions, integration patterns, and engineering lessons from building this multi-agent generative AI system, providing a reference implementation for developers incorporating language models and diffusion transformers into interactive applications.

Index Terms—Large Language Models, Diffusion Transformers, Game AI, Asset Generation, LoRA Fine-Tuning, Tool Use, Multi-Agent Systems, Procedural Content Generation

I. INTRODUCTION

The last five years have witnessed a dramatic transformation in what generative AI systems can produce. Large Language Models (LLMs) now reason about strategy, negotiate diplomatically, and invoke external tools with structured outputs. Diffusion Transformers (DiTs) generate high-quality images from natural language descriptions with remarkable prompt

adherence. Yet practical systems that combine these modalities into cohesive, playable applications remain rare. Most existing integrations either place LLMs in sandbox environments without competitive rules, or use diffusion models for concept art rather than production game assets with specific technical constraints.

StrategAI¹ is a full-stack strategy game—a Civilization-style 4X (explore, expand, exploit, exterminate) experience played on a square grid—that demonstrates how these technologies can work together in a single, functional system. The game features three AI civilizations, each controlled by an LLM with a distinct strategic persona, and generates all visual assets on demand using a DiT model running on workstation-class GPU hardware. The entire system is open-source and self-hosted, with no dependency on proprietary cloud services for asset generation.

A. Motivation

Independent game developers face two structural challenges that generative AI is uniquely positioned to address.

First, authoring competitive AI opponents traditionally requires behavior trees [1], finite state machines, or utility-based AI systems [2]. These approaches demand extensive manual authoring—designers must anticipate every strategic scenario and encode appropriate responses. As game complexity grows, the behavior tree becomes exponentially harder to maintain, and scripted AI inevitably exhibits exploitable patterns that players learn to circumvent. An LLM that reasons about strategy in natural language could adapt to novel situations without requiring every contingency to be pre-scripted.

Second, asset creation is the dominant cost in game development. A single strategy game might require hundreds of distinct visual assets across terrain types, building styles, unit classes, and leader portraits. Manual pixel art production for this volume of content is measured in artist-months. Procedural generation techniques—noise functions [3], grammar systems,

¹Source code is available at <https://github.com/maxtwotouch/StrategAI>. Video demonstrations: showcase at <https://youtu.be/h5gGrol2Log> and extended playthrough at <https://youtu.be/nvO3DZogazM>.

constraint solvers—can produce variation but rarely achieve the visual quality and stylistic coherence of hand-crafted art [4]. Generative image models have matured to the point where they can produce production-quality outputs, but integrating them into a reliable, automated pipeline with consistent style and perspective remains an engineering challenge.

StrategAI addresses both bottlenecks simultaneously: LLMs for strategic reasoning and diplomacy, and DiTs for on-demand pixel-art generation. The system demonstrates that these technologies are not merely research curiosities but practical tools accessible on consumer-grade hardware.

B. Problem Statement

This work investigates three interconnected research questions. First, can LLMs serve as autonomous strategic agents in a rule-bound competitive game, making decisions that are both strategically coherent and persona-consistent, without direct access to game state manipulation? Second, can diffusion models generate consistent, production-quality pixel-art assets across multiple asset families on workstation GPU hardware, meeting technical constraints such as transparent backgrounds, consistent perspective, and appropriate resolution? Third, can these two AI modalities be integrated into a cohesive, playable system with graceful degradation when either AI service is unavailable?

C. Contributions

This paper makes the following contributions:

- 1) **Intent-based LLM abstraction:** A layered architecture that decouples strategic reasoning (LLM-emitted intents) from tactical execution (deterministic Python resolution), enabling LLMs to control game entities without coordinate-level manipulation or rule validation.
- 2) **Four-layer prompt architecture:** A systematic approach to generative asset prompting that separates workflow configuration, style templates, semantic descriptions, and assembly logic across independently maintainable layers, producing consistent outputs across six asset families.
- 3) **Three-stage identity-preserving leader portrait pipeline:** A text-to-image (txt2img) and image-to-image (img2img) generation system that maintains character identity across splash art, profile portraits, and action scenes through carefully calibrated denoising parameters.
- 4) **LoRA fine-tuning methodology and public dataset:** A complete style-adaptation pipeline with a 3×2 experiment matrix evaluating caption detail against LoRA rank, trained on a curated 100-image dataset released on HuggingFace [5], with the resulting LoRA adapter also publicly available [6]. Systematic evaluation covers perspective consistency, style adherence, and trigger-token gating.
- 5) **Graceful degradation system:** A four-tier fallback architecture—generative, static, placeholder, and built-in—that ensures the game remains functional regardless of AI service availability.
- 6) **Reference implementation:** A fully functional, open-source codebase demonstrating practical patterns for multi-modal generative AI integration.
- 7) **TTS voiceover integration:** A game-start narration pipeline using OpenAI’s TTS API with theatrical narrator performance instructions, demonstrating prompt engineering applied to vocal delivery.

D. Paper Organization

Section II reviews related work in game AI, procedural content generation, model adaptation, and multi-agent systems. Section III presents the overall system architecture and technology rationale. Section IV details the LLM-driven strategic AI, including the intent abstraction, persona system, and memory management. Section V describes the generative asset pipeline, including the four-layer prompt architecture and leader portrait system. Section VI covers the LoRA fine-tuning methodology and experimental results. Section VII discusses frontend integration and user experience. Section VII-C within that chapter describes the TTS narration system as a third generative AI modality. Section VIII evaluates system performance and AI behavior quality. Section IX examines ethical considerations. Section X discusses limitations and future work. Section XI concludes.

II. RELATED WORK

A. Game AI and Large Language Models

Traditional game AI relies on hand-crafted decision systems. Behavior trees provide hierarchical task decomposition with clear execution semantics but require designers to anticipate every strategic branch [1]. Finite state machines offer predictable transitions between discrete behavioral modes but scale poorly as the number of states grows combinatorially with game complexity [2]. Utility-based AI systems evaluate candidate actions through weighted scoring functions, enabling more nuanced decision-making, but the weight tuning process is labor-intensive and the resulting behaviors can be opaque to designers.

The emergence of capable LLMs has prompted a wave of research into language-model-driven game agents. Park et al. [7] introduced generative agents—simulated characters that remember, reflect, and plan through natural language reasoning—but their system operated in a sandbox environment without competitive gameplay or rule enforcement. Wang et al. [8] developed Voyager, an LLM-powered agent that autonomously explores Minecraft through self-generated skill libraries and iterative code generation. Schick et al. [9] demonstrated with Toolformer that language models can teach themselves to invoke external tools via API calls—a capability that directly motivates our intent-based tool-use architecture.

StrategAI differs from prior work in two fundamental ways. First, our LLMs operate as competitive strategic agents in a rule-bound environment with explicit victory conditions, fog of war, and adversarial opponents—a substantially harder

setting than cooperative or sandbox environments. Second, our intent-based abstraction addresses a well-documented LLM weakness: while language models excel at high-level reasoning, they struggle with precise numerical calculations, spatial logic, and rule compliance. By separating strategic intent from tactical execution, we leverage LLM strengths while maintaining deterministic game mechanics.

B. Procedural Content Generation

Procedural Content Generation (PCG) has a long history in games, from the procedural dungeons of *Rogue* [10] to the vast procedurally generated universes of *No Man’s Sky* [11]. Traditional PCG techniques—Perlin noise for terrain [3], L-systems and shape grammars for structures, constraint solvers for level design—produce functional but aesthetically limited results. They excel at generating variation within parameterized spaces but struggle to produce content that matches the visual quality of hand-crafted assets [4].

Deep learning approaches to PCG initially focused on Generative Adversarial Networks (GANs), which demonstrated the ability to generate game levels and textures, but training instability and mode collapse limited practical adoption. Variational Autoencoders (VAEs) offered more stable training but produced characteristically blurry outputs.

Diffusion models [12], [13] have emerged as the state-of-the-art for image generation, offering high-quality, diverse outputs with strong prompt adherence. Peebles and Xie [14] advanced this paradigm by replacing the U-Net backbone with a pure Transformer architecture, creating Diffusion Transformers (DiTs) that exhibit superior scaling behavior. Liu et al. [15] proposed rectified flow, enabling substantially faster sampling—a technique adopted by the FLUX model family that powers our asset pipeline. However, the application of diffusion models to production game assets remains limited. Most existing work focuses on concept art generation rather than assets with specific technical requirements: transparent backgrounds, consistent perspective, tileable textures, and appropriate resolution for game engines.

C. Model Adaptation and LoRA

Full fine-tuning of large diffusion models—updating all 4 billion parameters of FLUX.2 Klein 4B, for example—is computationally prohibitive for independent developers. Parameter-Efficient Fine-Tuning (PEFT) methods address this by training a small number of additional parameters while keeping the base model frozen.

Low-Rank Adaptation (LoRA), introduced by Hu et al. [16], injects trainable low-rank decomposition matrices into the attention and convolution layers of a frozen model. At inference time, the base weights and LoRA weights are combined, enabling the adapted model to produce stylistically shifted outputs. LoRA adapters typically represent 0.1–1% of the base model’s parameter count, making them storage-efficient and trainable on a single GPU in hours rather than days.

Our contribution is threefold: we apply LoRA to the FLUX.2 Klein 4B DiT architecture, a newer model family

with rectified flow formulation and distilled inference; we systematically evaluate the interaction between caption detail level and LoRA rank through a 3×2 experiment matrix; and we validate trigger-token gating through style-leakage testing.

D. Multi-Agent Systems

Multi-agent systems (MAS) in games have traditionally employed homogeneous agents with shared decision logic [17]. Heterogeneous MAS, where agents exhibit different behavioral tendencies, are more complex to implement but enable richer, more naturalistic interactions.

StrategAI implements heterogeneous AI agents through persona-based prompting—a technique unique to LLM-driven agents. Each AI civilization receives a distinct character description that shapes its strategic decision-making while sharing the same underlying intent system and game engine. This approach achieves behavioral diversity without requiring separate AI implementations for each civilization.

III. SYSTEM OVERVIEW

A. High-Level Architecture

StrategAI comprises four principal components connected through well-defined service contracts. The Backend implements the game engine and LLM integration in Python using FastAPI. The Frontend, built with Next.js and React, renders the game state through an SVG-based map and manages all user interaction. The Asset Server, a separate FastAPI service, orchestrates ComfyUI-based image generation using FLUX.2 Klein 4B Distilled. The Training Pipeline, built on the Ostris AI Toolkit, produces the LoRA adapters that enforce a consistent top-down perspective across all generated assets.

A critical architectural principle governs cross-service communication: the Frontend mediates all interaction between services. The Backend and Asset Server never communicate directly. This mediation pattern simplifies each service’s responsibilities and enables independent development, testing, and deployment. (See Fig. 1 for a diagram of the system architecture and cross-service data flows.)

The game engine itself is organized into four layers that process decisions sequentially. At the top, the Strategic layer—the LLM, receiving a fog-filtered JSON view of the game world—emits high-level intents through OpenAI’s tool-use API. These intents flow down to the Operations layer, a set of deterministic Python functions that translate each intent into concrete goals, diplomatic actions, and production directives by querying the current game state for unit positions, distances, and rule constraints. The Tactical layer receives these goals and resolves them further: it runs A* pathfinding for movement goals, computes combat outcomes, and determines valid city-founding locations. Finally, the Validator layer checks every proposed action against the game rules—verifying turn order, movement budgets, diplomatic preconditions, and resource constraints—before any state mutation occurs. The Engine applies only validated actions, producing a new immutable GameState.

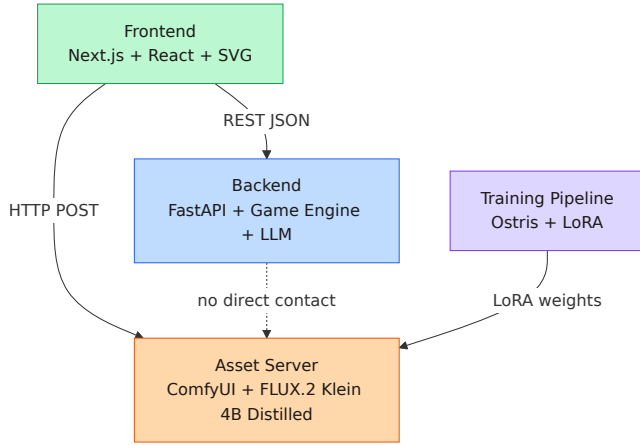


Fig. 1: System architecture showing four primary components and their data flows. The frontend mediates all communication—the backend and asset server never interact directly.

The key principle throughout this pipeline is that the LLM never accesses GameState directly. Every LLM intent passes through operations resolution, tactical execution, and rule validation before any state change. This four-layer separation ensures that strategic reasoning (where LLMs excel) is cleanly decoupled from tactical bookkeeping and rule enforcement (where deterministic code is necessary).

B. Technology Choices

Several technology decisions shaped the system’s capabilities and constraints.

FLUX.2 Klein 4B Distilled was selected as the image generation model over alternatives including Stable Diffusion XL and the larger FLUX.2 Klein 9B. Four criteria drove this choice: its Apache 2.0 license permits both research and commercial use; FP8 quantization reduces its VRAM footprint to approximately 8.4 GB, fitting within the memory budget of workstation GPUs; its 4-step distilled inference enables generation times that approach interactivity; and its DiT architecture with a Qwen 3 4B text encoder provides superior prompt adherence compared to CLIP-based models. Like all FLUX.2-family models, it does not support negative prompts—a consequence of classifier-free guidance being baked into the model weights during training rather than applied at inference time—so all prompt templates use positive-only, affirmative language.

GPT-5.4-mini was selected for strategic AI reasoning based on its tool-use capability, which enables structured intent emission through OpenAI’s function-calling interface, its 128K token context window, which accommodates game state serialization alongside rolling memory, its strategic reasoning quality relative to smaller models, and its production-grade API infrastructure with graceful error handling.

OpenAI TTS was selected for the game-start voiceover narration using the `gpt-4o-mini-tts` model with the cedar voice preset. The API’s `instructions` parameter

accepts a natural-language performance direction—here, a 60-word prompt specifying deep baritone, British theatrical accent, and dramatic pacing—enabling vocal delivery to be shaped through prompt engineering rather than audio post-processing.

ComfyUI serves as the inference orchestration layer, providing a node-graph workflow system with HTTP and Web-Socket APIs that enable programmatic control from the Asset Server [18].

C. Data Flow

The system operates through three primary data flows. The gameplay loop proceeds as follows: a human player submits an action through the Frontend, which sends it to the Backend API; the Backend validates the action and applies it to produce a new GameState; when the human ends their turn, the Backend iterates over AI civilizations, serializing each one’s fog-filtered local view into JSON, sending it to GPT-5.4-mini with the system prompt and history, receiving tool calls representing intents, and resolving those intents into validated actions.

The asset generation flow is initiated by the Frontend after receiving updated game state. The Frontend’s asset manifest resolver determines which assets are needed and fans out requests to the Asset Server. Each request triggers a ComfyUI workflow execution: the workflow JSON is patched with the appropriate prompt, a cryptographic seed is generated, and FLUX.2 Klein 4B Distilled performs 4-step distilled inference. The output image is post-processed (Lanczos resizing, background removal as needed), stored atomically, and its URL returned to the Frontend.

The training data flow is offline and unidirectional. A curated dataset of 100 top-down medieval pixel-art images is captioned at three detail levels. The Ostris AI Toolkit trains six LoRA variants against a frozen FLUX.2 Klein 4B base model, producing adapter weights deployed to the Asset Server’s ComfyUI models directory.

IV. LLM-DRIVEN GAME AI

The Backend implements three AI civilizations, each controlled by GPT-5.4-mini through OpenAI’s tool-use API. This section describes the architectural decisions that make this integration robust.

A. Intent-Based Abstraction

Giving an LLM direct control over game entities would be architecturally unsound. LLMs are probabilistic next-token predictors, not deterministic rule engines. They hallucinate identifiers, miscalculate distances, and propose actions that violate game rules. Even with careful prompting, expecting an LLM to consistently produce valid low-level game commands across hundreds of turns is unrealistic.

Our solution is a layered abstraction that separates strategic reasoning from tactical execution. The Strategic layer, implemented by the LLM, reasons about high-level goals: “Should I expand my territory, engage my neighbor, or consolidate my

defenses?” Its output is a set of structured intents—data objects like `Engage(target_civ_id=2)` or `Expand()`—that name strategic objectives without specifying how to achieve them. The Tactical layer, implemented as deterministic Python functions in the Operations module, resolves each intent into concrete goals by querying the current game state. (This layered abstraction—which the engine further decomposes into operations resolution, tactical execution, and validation sub-layers as described in Section III—keeps the LLM focused on high-level goals while deterministic code handles all mechanical bookkeeping.) Figure 2 illustrates this layered intent resolution pipeline.

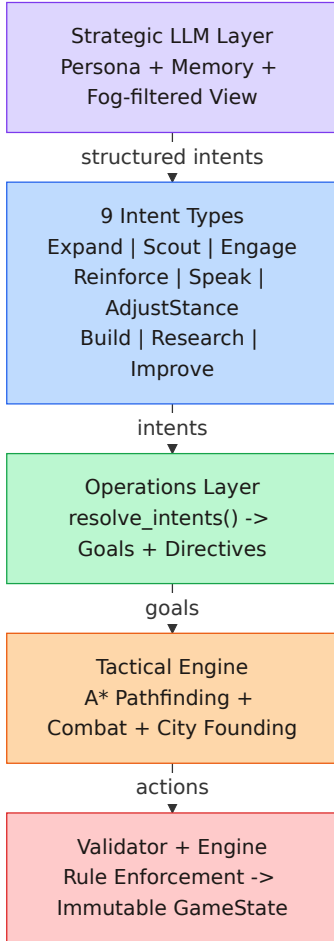


Fig. 2: Two-layer architecture separating strategic LLM reasoning from deterministic tactical execution. The LLM emits high-level intents; the engine resolves them into validated actions against an immutable GameState.

When the LLM emits `Engage(target_civ_id=2)`, the Operations layer automatically checks whether the civilizations are at war (declaring war if not), finds the closest military unit owned by the actor, locates the closest visible enemy unit or city belonging to the target, and emits `MoveTo` and `Attack` goals. The LLM never selects a unit ID, never computes a grid distance, and never needs to remember the rule that war

must be declared before attacking. These responsibilities are handled deterministically by code that cannot hallucinate.

This design represents a deliberate trade-off. The LLM loses the ability to execute fine-grained tactical maneuvers. In exchange, it gains robustness: every intent it emits will either produce a valid action or be rejected with a machine-readable error code that it can use to adjust its next decision. For a strategy game where the interesting decisions are strategic rather than tactical, this trade-off is appropriate.

B. Intent Types

The system defines nine intent types, each implemented as a frozen dataclass with type-annotated optional parameters. These nine types constitute the complete action vocabulary available to the LLM.

Expand instructs the engine to found a new city. The Operations layer identifies an available settler unit and searches for an optimal city site—an unoccupied, passable tile not adjacent to any existing city, prioritizing tiles with high food and production yields.

Scout directs a military unit toward unexplored territory. The Operations layer identifies the frontier—tiles at the edge of current visibility—and selects the closest reachable unexplored tile.

Engage initiates hostilities against another civilization. The Operations layer checks the diplomatic stance; if the civilizations are not at war, it emits a war declaration. It then identifies the closest military unit and enemy target.

Reinforce moves a military unit toward a friendly defensive position—either a specific city or a tile coordinate.

Speak sends a diplomatic message to another leader. The intent carries a message kind (chat, threat, offer peace, declare war, propose alliance, etc.) and free-form text that the LLM composes in its leader’s voice.

AdjustStance directly sets the diplomatic stance with another civilization to peace, war, or alliance.

Build appends a unit to a city’s production queue. The Operations layer validates that the requested unit type’s technology prerequisite is met.

Research sets the civilization’s active research target. The Operations layer validates that the technology is not already known and that all prerequisites are satisfied.

Improve orders a worker unit to construct a tile improvement—a farm, mine, or road. The Operations layer selects an idle worker and identifies a legal owned tile.

Each intent carries only semantic, high-level parameters. The LLM specifies target civilization IDs (not unit IDs), hints target tiles (not exact paths), and names technology IDs and unit types from enumerated lists provided in the serialized game view. This constrained vocabulary dramatically reduces the surface area for hallucination while preserving meaningful strategic choice.

C. Persona System

Three AI civilizations ship with the game, each defined by a civilization name, leader name, strategic traits, and a detailed persona prompt.

The **base system prompt** establishes the LLM’s role as an AI civilization leader, documents the structure of the serialized game state, explains each intent tool’s purpose, and encodes strategic principles. It also includes behavioral rules such as reading feedback from rejected intents and adapting strategy accordingly.

The **persona prompts** layer leader-specific motivations, voice, and constraints on top of this shared foundation. Genghis Khan of Mongolia is characterized as an aggressive expansionist who measures civilization by conquest, remembers every insult, and declares war within two turns of being threatened. Cleopatra of Egypt is a diplomatic opportunist who plays factions against each other, flatters potential allies, and avoids first strikes unless cornered. Gandhi of India is a peaceful developer who pursues science and culture victories, responds to threats with dignified protest, and welcomes alliances.

Each persona also includes explicit red lines—behavioral constraints that the LLM is instructed not to cross. These are not enforced by the engine but serve as prompt-level guardrails. The persona system achieves behavioral diversity without requiring separate AI implementations: all three leaders use the same nine intent types, the same Operations layer, and the same game engine. The only difference is the natural language description that shapes their strategic preferences.

D. Memory, Context, and Error Handling

LLMs are stateless: each API call is independent, with no inherent memory of previous turns. To enable coherent multi-turn strategic play, the system maintains rolling memory injected into the conversation context each turn.

Two logs feed the memory system. The intent log records the last 32 intents emitted by the AI, each annotated with its turn number and a summary of the outcome. The diplomatic message log records the last 32 messages—both sent and received—with sender, recipient, turn number, message kind, and full text. Both logs are filtered to the most recent 8 turns, ensuring memory remains relevant without accumulating stale information.

This memory serves three purposes. First, it prevents the AI from repeating strategies that the engine has already rejected. Second, it enables diplomatic continuity: the AI remembers who declared war, who offered peace, and who betrayed an alliance. Third, it maintains strategic coherence across turns, preventing the erratic goal-switching that can occur when an LLM lacks memory of its own past decisions.

Error resilience is implemented at multiple levels. If the OpenAI API is unreachable, the system catches the exception and returns empty decisions—the AI skips its turn rather than crashing the game. If the API returns malformed tool calls, the system parses each independently, skipping malformed ones and processing valid ones. Temperature configuration is omitted for models that do not support it.

The diplomacy system operates on a parallel track to the intent system. Relationship scores range from -100 (maximum hostility) to $+100$ (maximum alliance), with fixed deltas

applied per event type: declaring war costs 50 points, attacking costs 20, threatening costs 10, chatting gains 1, offering peace gains 8, accepting peace gains 25, proposing alliance gains 15, and accepting alliance gains 30. When peace is accepted, a 10-turn truce prevents either party from declaring war.

V. GENERATIVE ASSET PIPELINE

The Asset Server generates top-down medieval pixel-art assets on demand using FLUX.2 Klein 4B Distilled, orchestrated through ComfyUI.

A. Model and Infrastructure

FLUX.2 Klein 4B Distilled is a 4-billion-parameter Diffusion Transformer that replaces the convolutional U-Net backbone found in traditional latent diffusion models with a Vision Transformer operating on latent-space patches. This architectural shift provides a global self-attention receptive field that spans the entire latent representation, enabling superior long-range coherence.

The model employs rectified flow rather than conventional score-based diffusion. The forward process follows a straight-line path from data to noise, and the reverse process follows the same trajectory in reverse. This straight-line geometry enables efficient distillation: a 4-step student model can approximate the teacher’s 50-step trajectory in far fewer steps, with minimal quality degradation—roughly a twelvefold speedup.

FP8 quantization reduces the model’s VRAM footprint from approximately 16 GB to 8.4 GB with minimal perceptual quality loss, making it viable on workstation GPUs. The model uses a Qwen 3 4B text encoder rather than the CLIP encoder common in earlier diffusion models, providing stronger prompt adherence for the precise, technical descriptions required by game asset generation.

ComfyUI [18] provides the orchestration layer. Workflows are stored as version-controlled JSON node graphs. The Asset Server communicates with ComfyUI through its HTTP API, patching prompt text and random seeds into the workflow template, queuing the prompt, monitoring execution via WebSocket, and downloading the resulting image. Multi-node load balancing distributes requests across available ComfyUI instances.

B. Four-Layer Prompt Architecture

Generating consistent assets across six families with hundreds of combinatorial variants presents a prompt engineering challenge. Manual, ad-hoc prompt construction produces inconsistent results and couples generation logic to API implementation—any stylistic change requires code modifications. Our solution is a four-layer architecture that separates concerns into independently maintainable layers, delivering consistent output through templated prompt construction while decoupling the asset server API from the underlying prompt structure. (Figure 3 diagrams this four-layer prompt construction process.)

Layer 1: Workflow Configuration. The ComfyUI workflow JSON specifies the technical generation parameters that

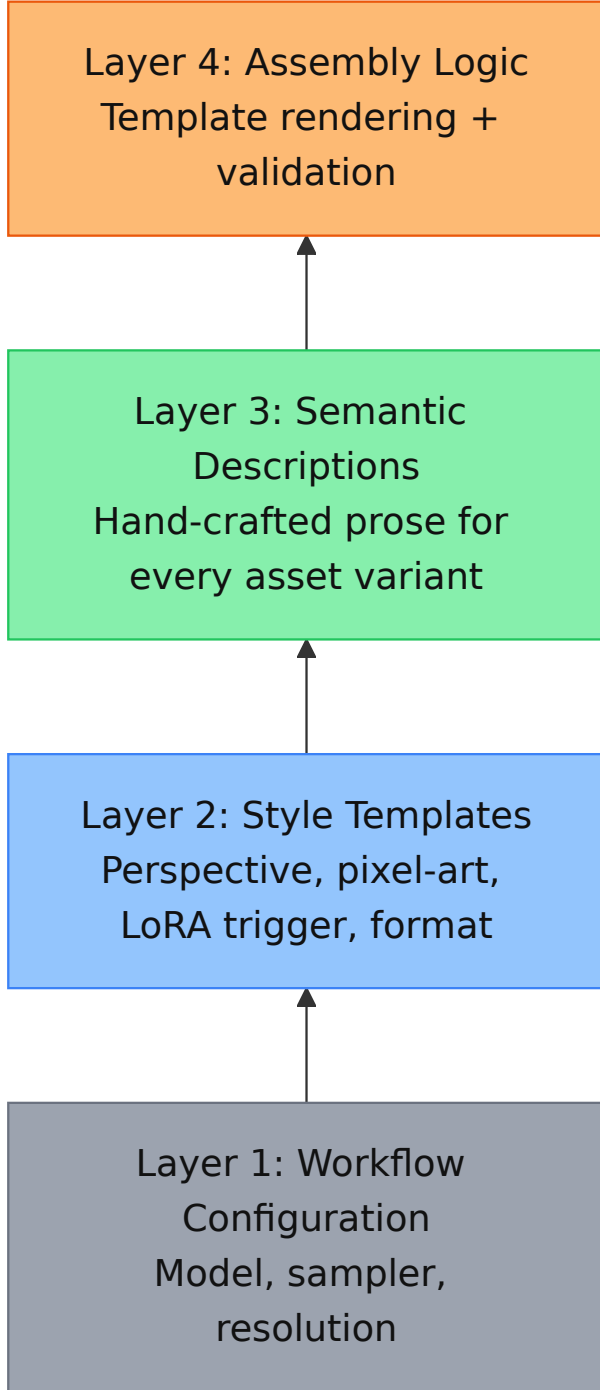


Fig. 3: Systematic prompt construction through four layers. Layers 1–2 (workflow and style) are version-controlled; Layer 3 exposes a curated vocabulary of 17 enum classes with 90+ semantic descriptions; Layer 4 assembles the final prompt.

are invariant across prompt variations: model file paths, sampler type and scheduler, inference step count (4), generation resolution (1024×1024), output resolution (256×256 via Lanczos resize), and post-processing configuration. This layer is version-controlled and validated at Asset Server startup.

Layer 2: Style Templates. The prompt templates stored in a centralized JSON configuration file define the stylistic constraints applied to each asset family. For tile assets, the template enforces top-down perspective via the LoRA trigger token `<tdp>` and the angle phrase “top-down view.”, specifies pixel-art aesthetics, and sets the output format. For leader portraits, the template uses cinematic language appropriate to splash screens. This separation means a style-level change requires editing only this single configuration file.

Layer 3: Semantic Descriptions. Seventeen enum classes define the combinatorial vocabulary for asset generation, each containing hand-crafted prose descriptions for every enum value—90+ across all families. For structures, the four enums are category (fortification, production, housing, sacred), style (Nordic wooden through Moorish), condition (pristine through fortified), and scale (small, medium, large). For leaders, the enums are archetype (8 values), culture (12 values spanning ancient Egyptian through Islamic Golden Age), time of day (6 values), and mood (8 values). This layer is the primary extension point.

Layer 4: Assembly Logic. Python prompt builder functions combine the preceding three layers. They load the appropriate style template, select the relevant semantic descriptions based on the client’s requested enum values, substitute these descriptions into the template’s placeholders, and validate that all required placeholders have been filled. The assembled prompt string is then injected into the ComfyUI workflow JSON and submitted for generation.

This four-layer separation yields several practical benefits. Style consistency is achieved because all assets within a family share the same template. Maintainability is improved because style changes require editing only Layer 2. Extensibility is straightforward because new asset types require only Layer 3 additions. It is worth noting that FLUX.2-family models do not support negative prompts (see Section III for the technical reason behind this constraint). All prompt templates therefore use positive-only, affirmative language.

C. Leader Portrait Pipeline

Leader portraits present a unique challenge: the same character must appear recognizably consistent across three different views—a dramatic full-screen splash image, a close-up profile portrait for UI displays, and an action scene showing the leader in a dynamic pose.

Our solution is a three-stage pipeline combining text-to-image (txt2img) and image-to-image (img2img) stages that chains generations to preserve identity. Throughout, we denote guidance scale as γ and denoise strength as δ . Figure 4 illustrates the pipeline with calibration sweet spots.

Stage 1: Splash Art (txt2img). A full txt2img generation at 1920×1088 resolution with denoise strength 1.0 and guidance

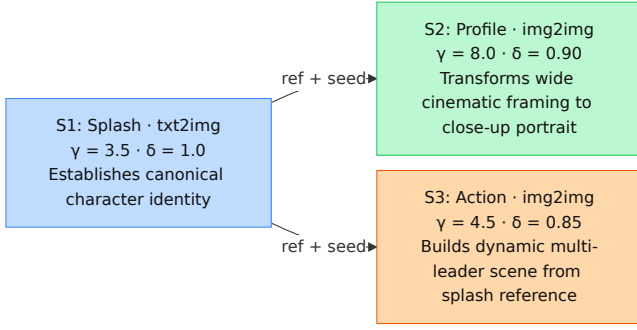


Fig. 4: Identity-preserving leader portrait generation through three stages with calibrated denoising parameters. Stage 1 establishes canonical identity (txt2img); Stages 2–3 use img2img with the splash as reference.

scale 3.5 establishes the canonical character appearance. The random seed is stored in the database, enabling exact reproduction.

Stage 2: Profile Portrait (img2img from Splash). The splash art is used as a reference image for a 512×512 square portrait with denoise strength 0.90 and guidance scale 8.0. The high guidance scale maximizes prompt adherence to preserve facial features, while the high denoise strength allows the composition change from wide cinematic framing to tight close-up. Empirical testing revealed that denoise values at or above 0.95 cause identity loss, while values at or below 0.80 prevent meaningful composition changes. The sweet spot at 0.90 preserves identity while enabling the framing shift.

Stage 3: Action Scene (img2img with reference stitching). The splash reference is combined with a fully transparent placeholder via image stitching within ComfyUI, then Lanczos-rescaled to 1920×1088 before VAE encoding. This stitch-then-encode approach produces a single coherent latent—avoiding the boundary artifacts that would arise from encoding two references separately—while the denoise strength of 0.85 and guidance scale of 4.5 allow dramatic compositional changes. For multi-leader scenes, the placeholder is replaced with a second leader’s reference image.

The combinatorial variety available through the leader enums is substantial: $8 \text{ archetypes} \times 12 \text{ cultures} \times 6 \text{ times of day} \times 8 \text{ moods}$ yields 4,608 possible prompt combinations for each of the three stages.

D. Multi-Mode Generation and Post-Processing

The Asset Server supports three generation modes, selectable per asset family through configuration. ComfyUI mode performs full AI generation using FLUX.2 Klein 4B Distilled with the LoRA adapter. Static mode serves pre-generated PNG files from a filesystem catalog. Placeholder mode generates simple colored rectangles with text labels using PIL.

Post-processing follows generation in ComfyUI mode. Images are generated at 1024×1024 —the minimum native resolution FLUX.2 Klein 4B supports—and Lanczos-downscaled

to 256×256 to minimize storage footprint while preserving the crisp edge definition essential for pixel-art aesthetics. Background removal produces transparent backgrounds for structures, objects, terrain features, and units; background tiles and leader portraits skip this step. All file writes use atomic semantics to prevent corruption.

The six asset families span a wide range of combinatorial variety. Structures support 420 possible combinations ($4 \text{ categories} \times 7 \text{ styles} \times 5 \text{ conditions} \times 3 \text{ scales}$). Nature objects support 140 combinations ($5 \text{ categories} \times 7 \text{ biomes} \times 4 \text{ seasons}$). Terrain elevation features support 75 combinations ($5 \text{ categories} \times 3 \text{ scales} \times 5 \text{ materials}$). Units support 4 types (archer, scout, settler, warrior). Background tiles support 4 types (water, grass, sand, stone). Leader portraits support thousands of combinations through their multi-enum parameter space.

VI. LORA FINE-TUNING FOR STYLE ADAPTATION

Prompt engineering alone on the base FLUX.2 Klein 4B model produces unreliable results for top-down perspective. The phrase “top-down view” yields a mixture of true orthogonal top-down, isometric, and 3/4-overhead perspectives depending on the random seed. For an automated game asset pipeline where every asset on a square grid must share the same camera plane, this inconsistency is unacceptable. This section describes our LoRA fine-tuning pipeline.

A. Dataset and Training Setup

The training dataset consists of 100 top-down medieval pixel-art images at 1024×1024 resolution, generated through a knowledge distillation pipeline: a FLUX.2 [dev] (32B) [19] teacher model with a Multi-Angles LoRA v2 [20] enforced top-down perspective at generation time, producing high-quality images that a FLUX.2 Klein 4B student model would learn to replicate through LoRA fine-tuning.

The dataset was manually curated from approximately 150–200 generated images down to the best 100. We focused primarily on structures (97 images) as the most visually complex and gameplay-critical asset family in a strategy game, with only 2 vegetation and 1 terrain image included as out-of-distribution probes. Structures came out clean and readable in the generation pipeline; vegetation blurred into indistinct masses after pixel-art quantization; rock formations rarely survived the strict top-down constraint with convincing geometry. The structure-dominated composition is methodologically advantageous: if the LoRA transfers top-down perspective to categories it barely saw during training (0% units, 3% terrain), the angle concept has been genuinely isolated from subject matter rather than merely memorized alongside architectural features.

Each image was captioned at three detail levels to support the experiment matrix. Detailed captions (50–100 words) describe materials, architectural features, condition, and context. Minimal captions (20–30 words) focus on essential characteristics. Ultra-minimal captions (5–10 words) specify only the asset type and pixel-art style designation. This three-level

design tests a core hypothesis: less text detail may produce less style binding, allowing the LoRA to learn a purer perspective concept that generalizes better to unseen asset types.

The trigger token `<tdp>` (top-down perspective) was chosen to be short, domain-agnostic, and unlikely to appear in natural language. At inference time, the token combined with the phrase “top-down view.” activates the LoRA adapter’s influence.

All six experiments shared a common training configuration: 2,500 training steps, batch size 1, adamw8bit optimizer, flowmatch scheduler, and checkpoint saves every 200 steps.

Both the curated dataset [5] and the six trained LoRA variants [6] are publicly released on HuggingFace under permissive licenses, enabling reproduction of our fine-tuning results and use of the style adapter in other FLUX.2-based pipelines.

B. Experiment Matrix and Methodology

The experiment matrix follows a 3×2 factorial design: three caption detail levels (detailed, minimal, ultra-minimal) crossed with two LoRA rank configurations (high: linear rank 128, conv rank 64; low: linear rank 64, conv rank 32). This design enables systematic evaluation of how caption granularity and model capacity interact.

The evaluation methodology uses eight validation prompts spanning in-distribution and out-of-distribution subjects. Two prompts target medieval structures (S1, S2)—the dominant in-distribution category comprising 97 of 100 training images. Two prompts target character units (U1, U2)—an asset family entirely absent from the training data, testing whether the LoRA’s top-down perspective generalizes to unseen subject categories. Three prompts target terrain and vegetation features (T1–T3), which appear only sparsely in the training set (3 images combined). A style-leakage probe (L1) places the trigger token alongside a modern sports car—semantically distant from the medieval training distribution—to verify that the LoRA’s influence does not contaminate anachronistic subjects.

C. Results and Deployed Checkpoint

Prior to LoRA fine-tuning, we evaluated several base models for their ability to produce consistent top-down perspective through prompting alone. Stable Diffusion XL [21], FLUX.1 [dev], and FLUX.2 [dev] [19] all failed to achieve reliable camera positioning—the phrase “top-down view” produced a mixture of orthogonal, isometric, and oblique angles varying unpredictably with the random seed. Even FLUX.2 [dev] augmented with the Multi-Angles LoRA v2 [20] exhibited angle discrepancies across generations, as the adapter was designed for 72 discrete camera positions rather than strict overhead enforcement. Our custom-trained LoRA, by contrast, was optimized specifically for consistent top-down perspective and enforces this constraint reliably across all generated assets, as demonstrated below.

Qualitative evaluation of the complete results grid yields several key findings. Figure 5 presents a representative sample from this evaluation grid.

The base model without LoRA cannot reliably enforce top-down perspective. Text prompts alone produce a mixture of perspectives including isometric and oblique angles, with the specific angle varying by random seed. This confirms the necessity of fine-tuning.

All six LoRA variants enforce consistent top-down perspective, reliably producing overhead camera angles across in-distribution structures and out-of-distribution subjects.

Caption detail and rank interact in a fidelity–generalization trade-off. The detailed-high variant (detailed captions, high rank) achieves maximum in-distribution architectural fidelity—structures rendered with crisp crenellations, visible timber framing, and distinct stone masonry courses—at the cost of slightly reduced out-of-distribution generalization. The ultra-low variant achieves the strongest generalization to unseen asset types but produces less architecturally specific structures.

Training dynamics reveal distinct regimes. Early checkpoints (steps 500–1,000) produce noisy outputs. The sweet spot emerges around steps 1,500–2,000, where perspective consistency and style adherence peak simultaneously. Beyond step 2,200, some experiments exhibit overfitting artifacts: rigid repetition of training-set layouts, color palette collapse, and reduced compositional diversity.

Style leakage was evaluated using a probe that includes the `<tdp>` trigger token alongside a semantically distant modern subject—a red sports car. This tests whether the LoRA’s medieval styling contaminates anachronistic subjects when the trigger is active. The detailed-high variant showed the cleanest separation: glossy red paint, aerodynamic curves, and contemporary design language remained intact with minimal medieval palette shift. In contrast, ultra-high, minimal-low, and minimal-high exhibited subtle art-style mimicry—earthen color tones and faint surface texturing characteristic of the training distribution. This confirms that style leakage exists on a spectrum modulated by caption detail and rank configuration, and that detailed-high at step 1,800 provides the best balance of perspective enforcement and domain-appropriate restraint.

The detailed-high variant at step 1,800 was selected for deployment. This checkpoint achieves maximum in-distribution architectural fidelity while maintaining acceptable out-of-distribution generalization and avoiding overfitting artifacts. The deployed LoRA file is 353 MB (high-rank configuration: linear rank 128, convolution rank 64) and loaded at strength 1.0. Lower-rank variants in the experiment matrix ranged from 89 MB (ultra-low, rank 32/16) to 177 MB (low, rank 64/32), with file size directly reflecting the rank–capacity–generalization trade-off explored in the 3×2 design.

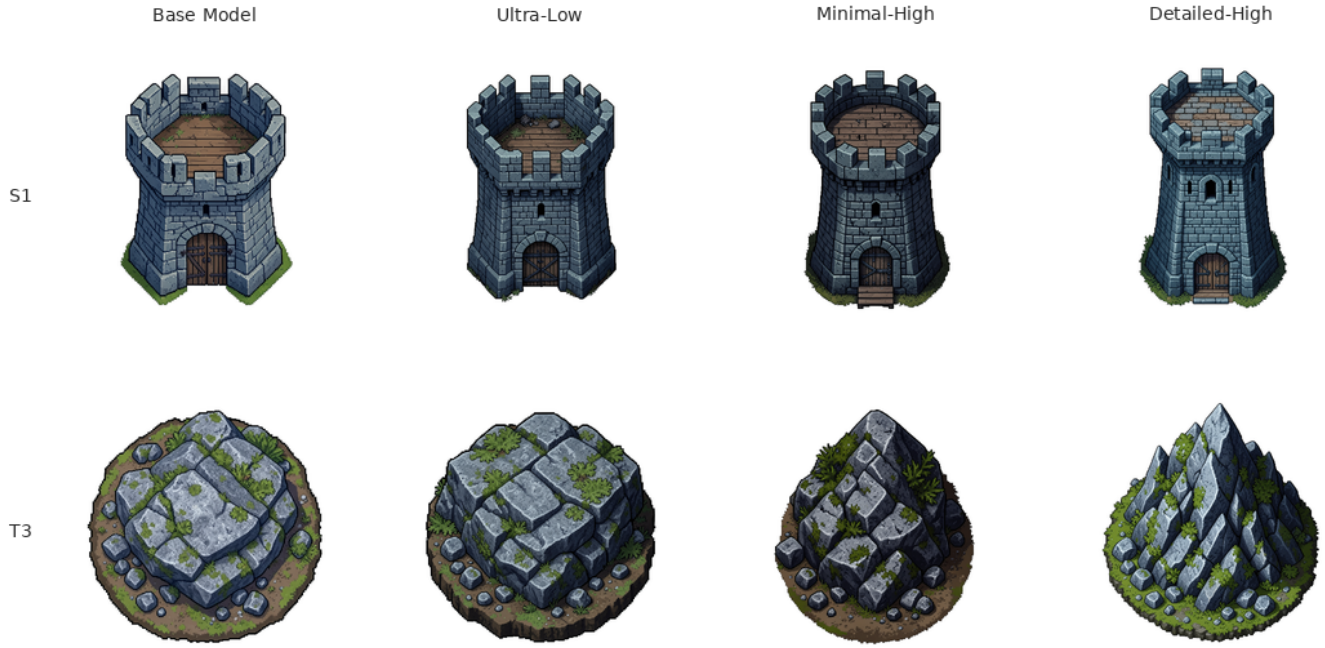


Fig. 5: Qualitative comparison of LoRA caption detail strategies. A 2x4 grid showing two prompt subjects (watchtower, boulder) across four conditions: base model (no LoRA), ultra-low, minimal-high, and the deployed detailed-high variant. Each LoRA variant is shown at its own optimal checkpoint.

VII. FRONTEND INTEGRATION

The Next.js frontend binds the Backend game engine and Asset Server into a cohesive player experience.

A. Asset Resolution and Graceful Degradation

When the game state updates, the frontend’s asset manifest resolver determines which generative assets are needed and fans out requests to the Asset Server. Mapping tables translate game entities to asset server parameters: engine terrain types are mapped onto the asset server’s background tile types, game unit types map to the asset server’s unit categories, and each AI leader’s traits map to architectural styles for their civilization’s structures.

The resolver implements four-tier fallback, attempted in order. Tier 1 requests full AI-generated assets from the Asset Server in ComfyUI mode—the highest quality, requiring GPU inference. If the Asset Server is unreachable, Tier 2 falls back to static pre-generated PNG files served from the Asset Server’s filesystem catalog, providing consistent quality without GPU dependency for common asset configurations. If static assets are unavailable, Tier 3 uses placeholder mode—colored rectangles with text labels generated by the Python Imaging Library, functional but aesthetically minimal. If the Asset Server is completely unreachable, Tier 4 provides built-in fallbacks: color-coded tiles from a hardcoded terrain color palette, Unicode glyph characters for units, and initial letters for leader portraits. This four-tier design ensures the game

remains playable—with progressively reduced visual quality—regardless of which AI services are available.

B. Diplomatic Interface

The diplomacy system provides a chat-based interface where human players compose free-text messages to AI leaders, select a message type, and receive LLM-generated responses in the leader’s characteristic voice. The AI’s response reflects its persona, the current diplomatic relationship score, and the conversation history. The relationship tracking display shows each AI leader’s stance, a numeric relationship score from -100 to $+100$, and a history of recent diplomatic events.

C. TTS Narration

A third generative AI modality—text-to-speech—provides the player’s first encounter with AI-generated content. When the player clicks “Begin Campaign,” the frontend constructs a five-sentence epic narration script parameterized by the player’s civilization and leader names, then sends it to the backend’s POST `/audio/intro` endpoint. The backend calls OpenAI’s TTS API (`gpt-4o-mini-tts` model, `cedar` voice) with a set of narrator performance instructions that shape the vocal delivery: a deep, commanding baritone with a refined British theatrical accent, slow solemn pacing, and long dramatic pauses between clauses. The returned MP3 audio plays over the ambient intro music with automatic volume ducking (music reduced to 18% volume during narration), producing an immersive “voice of history” opening sequence. A “Replay Proclamation” button lets the player

re-trigger generation, and a persistent mute toggle stores its state in `localStorage`. The feature degrades gracefully: if the `OPENAI_API_KEY` is not configured, the endpoint returns HTTP 503 and the intro screen falls back to a silent presentation.

This integration demonstrates a practical pattern for prompt engineering applied to vocal performance. The narrator instructions—a 60-word prompt specifying accent, register, pacing, emotional arc, and constraints against impersonating real figures—produce consistently theatrical delivery across varied civilization and leader names. The dual `HTMLAudioElement` architecture (separate elements for music and narration) enables independent volume control without a Web Audio API dependency. The frontend audio hook implements a three-phase state machine (silent $\rightarrow$ intro $\rightarrow$ ambient) with crossfade transitions ranging from 650 ms to 2,400 ms, managed entirely through `requestAnimationFrame` volume ramping.

VIII. EVALUATION

We evaluate StrategAI across three dimensions: system performance, AI behavior quality, and asset quality. Where quantitative measurements are available, we report them with appropriate caveats. Where only qualitative assessment is possible, we describe observed patterns and acknowledge the limitations of subjective evaluation.

A. System Performance

LLM decision latency is the dominant factor in turn-processing time. Each AI civilization’s turn requires a round-trip to the OpenAI API, typically taking 2–4 seconds per AI civilization. This latency is acceptable for turn-based gameplay but would be prohibitive for real-time applications.

Asset generation times were informally observed during development rather than measured through systematic benchmarking. On a Blackwell RTX 6000 with FP8 precision, simple assets generated in approximately 2.5–4 seconds, while complex leader portraits required 4–6 seconds. On an RTX 3090, the range shifted upward to approximately 3.5–7 seconds. These observations should be interpreted as rough guidance rather than rigorous performance data. An estimated cache hit rate based on expected gameplay patterns suggests that caching substantially reduces redundant generation in practice, but this estimate has not been empirically validated.

Frontend rendering performance was observed to maintain 60 frames per second on maps of up to approximately 1,000 visible tiles with SVG-based rendering.

B. AI Behavior Quality

Qualitative observation across multiple playthroughs reveals distinct strategic patterns aligned with the designed personas. Genghis Khan consistently pursued aggressive expansion and rapid war declaration. Cleopatra pursued diplomatic strategies: forming alliances and playing factions against each other. Gandhi prioritized peaceful development: early technology research and active pursuit of alliances.

Emergent behaviors included opportunistic attacks on weakened neighbors, the formation of defensive alliances against an aggressive Mongolia, and revenge behavior where Genghis Khan would declare war on civilizations that had previously attacked him. Observed limitations include occasional repetition of rejected strategies, suboptimal city placement in edge cases, and difficulty coordinating simultaneous multi-unit attacks. We acknowledge the absence of quantitative metrics for AI behavior quality; the observations reported here are qualitative and subjective, based on developer playtesting.

TABLE I: Qualitative comparison of AI leader behavior patterns across gameplay phases.

	Genghis Khan	Cleopatra	Gandhi
Persona	Aggressive expansionist	Diplomatic opportunist	Peaceful developer
Dominant Intents	Engage, Expand, Reinforce	Speak, Adjust-Stance, Build	Research, Build, Improve
Early Game	Military unit production	Alliance formation	Technology research
Mid Game	Sustained warfare	Faction manipulation	Defensive buildup
Late Game	Conquest push	Economic dominance	Science/culture victory

C. Asset Quality

Visual inspection reveals several patterns. Style consistency is achieved across asset families: structures, objects, terrain features, and units all exhibit the same pixel-art aesthetic with crisp edges and limited color palettes. The LoRA adapter’s enforcement of top-down perspective is reliable—generated assets consistently show the subject from directly above.

The 420 possible structure combinations produce visually distinct results. The seven architectural styles produce meaningfully different visual outputs: Nordic wooden structures differ visibly from Moorish arched designs. Leader portraits preserve character identity across the three pipeline stages, achieved through the pipeline’s use of the splash art as reference. We acknowledge the absence of automated quantitative metrics such as Fréchet Inception Distance (FID) or CLIP score for asset quality.

IX. ETHICAL CONSIDERATIONS

The integration of generative AI technologies into interactive entertainment raises ethical considerations spanning bias, environmental impact, intellectual property, and reproducibility.

A. Bias in Generated Content

StrategAI employs three AI civilizations modeled after historical figures. The use of culturally specific historical figures carries inherent risks of stereotyping and reductive characterization. LLMs trained on internet-scale corpora have been shown to amplify stereotypes related to nationality, gender, and ethnicity [22], and the persona prompts, while carefully

written, are mitigations layered on top of a base model whose training distribution may contain biased representations.

The diffusion-based asset pipeline introduces additional representation concerns. The curated training dataset of 100 medieval pixel-art images is dominated by European medieval architectural styles [23], which means generated assets for non-European civilizations may exhibit stylistic incongruity. This is an acknowledged limitation of the current dataset composition.

B. Environmental Impact

Asset generation using FLUX.2 Klein 4B Distilled consumes approximately 0.21–0.58 watt-hours per image on workstation GPUs, based on observed generation times and typical GPU power draw [24]. The LoRA training process required approximately 2 hours per experiment, totaling approximately 12 GPU-hours for the complete experiment matrix. FP8 quantization halves the model’s VRAM requirement and correspondingly reduces inference energy. Caching reduces redundant generation of identical asset configurations. The rolling memory window of 8 turns prevents unbounded growth of LLM context, constraining token consumption.

C. Copyright and Intellectual Property

The legal status of AI-generated content remains unsettled across jurisdictions. The training dataset for the LoRA adapter was generated using FLUX.2 [dev] (32B) as a teacher model and curated by the StrategAI team. The copyright status of AI-generated game assets is unclear: under current U.S. Copyright Office guidance, works created entirely by artificial intelligence without sufficient human creative input are not eligible for copyright protection [25]. Whether the human creative choices in the four-layer prompt architecture satisfy the originality requirement is uncertain [26]. The FLUX.2 Klein 4B Distilled base model is released under Apache 2.0, a deliberate selection criterion.

D. Reproducibility

The system uses cryptographically strong random seeds stored in the database. Given the same model weights and ComfyUI configuration, any generated image can be exactly reproduced. The LoRA training pipeline fixed random seeds across experiments. However, GPT-5.4-mini is accessed through OpenAI’s cloud API—model snapshots change over time, so LLM behavior may not be exactly reproducible. The intent-based abstraction partially mitigates this by constraining LLM behavior to nine predefined actions.

X. LIMITATIONS AND FUTURE WORK

A. Architectural Limitations

The Backend uses an in-memory game store, restricting deployment to a single process. The Asset Server holds HTTP connections open for the duration of image generation, limiting concurrent request throughput. The Frontend mediates all cross-service communication, creating a single point of coordination.

B. AI Limitations

Multi-turn strategic planning is limited: the LLM reasons reactively, responding to the current game state, rather than proactively pursuing long-term objectives. AI civilizations act independently with no coalition logic. Human players can learn to exploit AI behavioral patterns.

C. Future Directions

Persistent storage using PostgreSQL would enable game state persistence and multi-instance deployment. Locally hosted open-weight LLMs would eliminate the cloud API dependency. Multi-agent coalition logic would enrich diplomatic gameplay. Automated quantitative evaluation metrics—FID and CLIP score for asset quality, win-rate analysis for AI behavior—would replace the current reliance on qualitative assessment. Expanded asset types including character animations, environmental sound effects, and procedural narrative generation would broaden the generative pipeline’s scope.

XI. CONCLUSION

StrategAI demonstrates that generative AI technologies—Large Language Models for strategic reasoning and Diffusion Transformers for asset generation—can be integrated into a cohesive, functional game system on workstation-grade hardware. Three architectural patterns underpin this integration.

First, intent-based abstraction bridges the gap between LLM reasoning and game-engine execution. By constraining LLM output to nine high-level intents and resolving those intents through deterministic operations, the system leverages language models’ strategic reasoning capabilities while maintaining the rule compliance and numerical precision that games require. The LLM never selects unit identifiers, never computes grid distances, and never validates game rules—responsibilities that remain with deterministic code—yet retains meaningful strategic agency over expansion, warfare, diplomacy, and development.

Second, the four-layer prompt architecture combined with LoRA fine-tuning enables consistent, on-demand asset generation across six diverse families. The trigger token mechanism provides selective control, ensuring the adapter’s influence activates only for tile assets while leaving leader portraits and background tiles unaffected.

Third, graceful degradation through four-tier fallback ensures the system remains functional regardless of AI service availability. From full GPU-accelerated generation down to built-in color codes and glyph characters, the game degrades progressively rather than failing catastrophically.

The limitations are real: cloud API dependency undermines reproducibility, qualitative evaluation lacks the rigor of quantitative metrics, and the single-instance architecture constrains deployment. But the integration patterns demonstrated here—intent abstraction, layered prompt engineering, parameter-efficient fine-tuning, and multi-tier fallback—provide a practical roadmap for independent developers seeking to harness generative AI in their own projects. As models continue to

improve in quality, speed, and efficiency, these patterns will only become more accessible and more powerful.

REFERENCES

- [1] C. Simpson, “Behavior trees for AI: How they work,” Gamasutra (later Game Developer), 2014.
- [2] S. Rabin, *Game AI Pro: Collected Wisdom of Game AI Professionals*. CRC Press, 2013.
- [3] K. Perlin, “An image synthesizer,” *ACM SIGGRAPH Computer Graphics*, vol. 19, no. 3, pp. 287–296, 1985.
- [4] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [5] S. Asakskogen, “topdown-medieval-pixelart: A curated top-down medieval pixel-art dataset,” <https://huggingface.co/datasets/stixxert/topdown-medieval-pixelart>, 2026.
- [6] —, “strategai-topdown-medieval-style-lora: LoRA adapters for FLUX.2 Klein 4B top-down pixel-art generation,” <https://huggingface.co/stixxert/strategai-topdown-medieval-style-lora>, 2026.
- [7] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [8] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] M. Toy, G. Wichman, K. Arnold, and J. Lane, “Rogue,” Version Unix, 1980.
- [11] Hello Games, “No man’s sky,” Video Game, 2016.
- [12] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [13] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [14] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [15] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.
- [16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.
- [17] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.
- [18] ComfyUI, “ComfyUI: A powerful and modular stable diffusion GUI and backend,” <https://github.com/comfyanonymous/ComfyUI>, 2023.
- [19] Black Forest Labs, “FLUX.2 Klein 4B distilled,” Model card. <https://huggingface.co/black-forest-labs/FLUX.2-klein-base-4B>, 2025, accessed: 2026-05-30.
- [20] L. Odin, “Flux-2-Multi-Angles-LoRA-v2: 72 camera angle positions for FLUX.2,” LoRA adapter. <https://huggingface.co/lovix93/Flux-2-Multi-Angles-LoRA-v2>, 2025.
- [21] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach, “SDXL: Improving latent diffusion models for high-resolution image synthesis,” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [22] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?” in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.
- [23] K. Crawford and T. Paglen, “Excavating AI: The politics of images in machine learning training sets,” <https://excavating.ai>, 2019.
- [24] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.
- [25] P. Samuelson, “Generative AI meets copyright,” *Science*, vol. 381, no. 6654, pp. 158–161, 2023.
- [26] A. Hertzmann, “Can computers create art?” *Arts*, vol. 7, no. 2, 2018.